Diffraction Simulation Coding Coursework
Word Count = 2567

James Barnes

25^{th} November 2025

1 Abstract

I successfully simulated and graphed both Fresnel and Fraunhofer diffraction from a square aperture in 1D and 2D using dblquad, and from a circular aperture in 2D using dblquad and using the Monte Carlo method. All simulations behaved as expected, following physical expectations, with the parameters included in the figure title, and so the project was successful. To make the viewing process more convenient, I also implemented a series of questions to navigate, so only the appropriate plot is generated, saving time.

2 Introduction

The task was to simulate different types of diffraction; Fresnel (near-field) and Fraunhofer (far-field), using various methods, dimensions, and aperture shapes. Scipy's "dblquad" function was used to integrate the Fresnel equation, both in 1D and 2D, where the aperture shape could be varied by adjusting the limits of the integration (in our case we used a circle so only the "y" limits). To test other methods, the Monte-Carlo method was also implemented for a circular aperture rather than dblquad, so that they could be compared.

3 Theory

The Fresnel equation was given to us in the exercise 2 worksheet, and describes how the electrical field diffracts onto a screen at distance z from the aperture;

$$E(x,y,z) = \frac{kE_0}{2\pi z}\int_{Aperture} e^{\frac{ik}{2z}[(x-x')^2+(y-y')^2]}dx'dy' \tag{1}$$

where x, y are the coordinates of the screen, x', y' are the coordinates of the aperture, k is $2\pi/\lambda$ and E_0 is the Electric field incident to the aperture. However a problem arises when we try to call dblquad, as it cannot integrate imaginary numbers, and so by splitting the equation into two integrals (Real

and Imaginary components), then integrating them separately and conjoining them after, we can circumnavigate the issue;

$$E(x,y,z) = \frac{kE_0}{2\pi z}\int_{x_1'}^{x_2'}\int_{y_1'(x')}^{y_2'(x')}\cos\left(\frac{k}{2z}[(x-x')^2+(y-y')^2]\right)\,dy'\,dx'$$

$$+\ i\frac{kE_0}{2\pi z}\int_{x_1'}^{x_2'}\int_{y_1'(x')}^{y_2'(x')}\sin\left(\frac{k}{2z}[(x-x')^2+(y-y')^2]\right)\,dy'\,dx'. \quad (2)$$

Then once we have successfully integrated to find the electric field, we can find the intensity at a given screen dimension I,

$$I(x,y,z) = \epsilon_0 c[Re(E(x,y,z))^2 + Im(E(x,y,z))^2] \quad (3)$$

where ϵ_0 is the permittivity of free space, and c is the speed of light in a vacuum.

Finally, when we want to use the Monte-Carlo method, we use the equation;

$$\int f dA \approx A(< f > \pm\sqrt{\frac{< f^2 > - < f >^2}{N}}) \quad (4)$$

where $< f >$ is the average value of the function, N is the number of N points used, A is the area bound by the function, and $< f^2 >$ is the average of the function squared. Now we have every equation we need to go on with the programming.

4 Explanation of Code

4.1 Importing Libraries and Defining the Functions

First of all there are some important libraries that we need to import; scipy for our dblquad function and physical constants, numpy for our equations and random number generation, matplotlib.pyplot for our plots, sys sot that the user can easily exit the program whenever they like, and time so we can record how long each plot took to generate to be used for analysis.

Listing 1: Importing Libraries

```
from scipy import integrate
import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import epsilon_0 as e0
from scipy.constants import c
import sys
import time
```

Following the libraries, we must define the functions we plan on calling. Starting with our Fresnel equation, which we split into the real (Fresnel2dreal) and imaginary (Fresnel2dimag) components, where the real component is a function of sine, and the imaginary component is a function of cosine.

Listing 2: Defining Fresnel Equation

```
def Fresnel2dreal (yp, xp, y, x, k, z):
    E0 = 1 #V/m
    Fresnel2dreal = ((k * E0)/(2 * np.pi * z)) * np.cos( (k)
        /(2 * z) * ( (x - xp)**2 + (y - yp)**2 ) ) #V/m

    return Fresnel2dreal

def Fresnel2dimag (yp, xp, y, x, k, z):
    E0 = 1 #V/m
    Fresnel2dimag = ((k * E0)/(2 * np.pi * z)) * np.sin( (k)
        /(2 * z) * ( (x - xp)**2 + (y - yp)**2 ) ) #V/m

    return Fresnel2dimag
```

Now we need to define our functions for the y limits of our integration, when we consider a circular aperture, using R as the radius of the aperture.

Listing 3: Circular Aperture y Limits

```
def yminfunc (xp):
    return - np.sqrt( R**2 - xp**2 )

def ymaxfunc (xp):
    return np.sqrt( R**2 - xp**2 )
```

Lastly, we want our functions for the Monte-Carlo method. First we want the polar form of the Fresnel equation ($Fresnel2d_{Mc}$),since we aren't integrating so we can keep the equation together. We also want our actual Monte-Carlo function, which uses the mean value of $Fresnel1d_{Mc}$ multiplied by the area of the square length R to find an estimate for the integral (given in the coding exercise 2 sheet). We use a square for now since it's difficult to generate random x, y samples within a circle, and so we generate them in a square and ignore all values which lie outside the region of our circular aperture (this is done by multiplying $Fresnel2d_{Mc}$ by a boolean array that filters for inside the region). The mean of our function, and our errors are calculated using their general equations.

Listing 4: Monte-Carlo Functios

```
def Fresnel2d_Mc (yp, xp, y, x, k, z):

    E0 = 1 #V/m
    return ((k * E0)/(2 * np.pi * z)) * np.exp( (1j * k)/(2
        * z) * ( (x - xp)**2 + (y - yp)**2 ) ) #V/m
```

```
def MC (Fresnel2d_Mc, xmin, xmax, ymin, ymax, xp, yp, k, z,
    N):

    x_samples = np.random.uniform(low=xmin, high=xmax, size=
        N)
    y_samples = np.random.uniform(low=ymin, high=ymax, size=
        N)

    r2 = x_samples**2 + y_samples**2
    boolean_array = np.less_equal(r2, R**2)

    values = Fresnel2d_Mc(y_samples, x_samples, yp, xp, k, z
        )
    values = values * boolean_array

    mean = values.sum() / N
    meansq = (values * np.conjugate(values)).sum() / N

    results = a * mean
    results_error = a * np.sqrt( (meansq - mean * mean) / N
        )
    return (results, results_error)
```

4.2 Choosing Between Fresnel and Fraunhofer Diffraction

To make viewing the plots more convenient/efficient, we create an option between Fresnel (near-field) and Fraunhofer (far-field) diffraction so we don't waste time generating everything at once. I do this by creating a while loop which stays active constantly so that we can return to the beginning just by breaking a later while loop. Then I set the necessary parameters for each diffraction type based on which option is picked. If an invalid option is selected (not q) then I return an error message and wait for a valid choice, also if the user wishes to quit the program then they can press q to quit. After each option is selected a create a new line in the console by using n n, to make it more readable, and the crrent while loop breaks so the program can continue.

Listing 5: Choosing Between Fresnel and Fraunhfer

```
while True:
    MyInput = '0'
    while MyInput != "q":
        MyInput = input('Enter a choice, "1" for near-field
            (Fresnel) case, or "2" for far-field (Fraunhofer)
             case, or "q" to quit.')
        if MyInput == '1':
            print('You have selected "1": near-field (
                Fresnel) case\n') #"\n" creates a space in
```

```
                the console text to make it easier to read

            z = 0.005 # /m
            ymin_s, ymax_s = -2.5e-4, 2.5e-4 # /m
            xmin_s, xmax_s = -2.5e-4, 2.5e-4 # /m
            xmin, xmax = -1e-4, 1e-4 # /m
            ymin, ymax = -1e-4, 1e-4 # /m
            R = 1e-4 # /m
            break
#============================================================

        elif MyInput == '2':
            print('You have selected Part 2: far-field (
                Fraunhofer) case\n')

            z = 0.05 # /m
            ymin_s, ymax_s = -2.5e-3, 2.5e-3 # /m
            xmin_s, xmax_s = -2.5e-3, 2.5e-3 # /m
            xmin, xmax = -1e-5, 1e-5 # /m
            ymin, ymax = -1e-5, 1e-5 # /m
            R = 1e-5 # /m
            break
#============================================================
        elif MyInput != 'q':
            print('This is not a valid choice\n')
#============================================================
        elif MyInput == 'q':
            print('Program ending')
            sys.exit() #exit program
#============================================================
```

4.3 Defining Set Parameters

Here we define the parameters that don't depend on whether Fresnel or Fraunhofer is used, so we make the code more clean by putting it here rather than each if/elif independently. These parameters include N (Number of random numbers for Monte-Carlo to generate), a (the area of our square aperture), w (aperture width) which is useful information for both square and circular apertures, $xlengthsteps, ylengthsteps$ (the number of divisions in the axes of our plot), y, x (arrays containing our x and y axes for our plot), k (a function of the wavelength which remains constant throughout all simulations), $real_{results}, imag_{results}, realerror_{results}, imagerror_{results}$ (lists to store our results from dblquad), and E, E_{error} (our lists to store results from Monte-Carlo).

Listing 6: Set Parameters

```
N = 1000000

a = (xmax - xmin) * (ymax - ymin) # /m^2
```

```
w = (xmax - xmin) # /m

xlengthsteps = 200 #For x
ylengthsteps = 200 #For y

y, x = np.linspace(ymin_s, ymax_s, ylengthsteps), np.
    linspace(xmin_s, xmax_s, xlengthsteps) # /m

k = ( 2 * np.pi ) / (500e-9) # /m

real_results = []
imag_results = []
realerror_results = []
imagerror_results = []
E = []
E_error = []
```

4.4 Integrating with method of choice

Now that all our parameters have been set, we can finally choose which part from the exercise we want displayed (1D, 2D square, 2D and Monte Carlo circle are our four options, each corresponding to a "part" from the exercise). Similarly to the option between Fresnel and Fraunhofer, a while loop is created, where $MyInput$ is reset to zero. The the user is then presented with a choice for each part, with each input 1-4 corresponding to each one. Depending on which part is selected, a message is printed confirming this, and the method is performed (MC for Monte-Carlo or Dblquad for the others) albeit with a few variations depending on which one specifically is selected. For example, for 1D, y must be set to zero within the for loop so in each iteration it isn't considered (this means that we must recreate our axes in 2D parts since if 1D is selected first, the $y = 0$ will mess up the others), or for parts 3 and 4 the aperture dimensions must be reset to the radius of the aperture. Also after any part is selected, $Part$ is set to 1-4 depending on the one selected so we can keep track of which method to plot, and before/after each loop, $start_{time} and stop_{time}$ are set to the current time ($time.time()$) so that we can record how long each method took to complete. The actual method includes our integration method (dblquad or MC) inside a for loop, iterating for each x and y value and appending the values to our results lists defined above, and once this is completed the while loop breaks so the program can continue (like the last one).

Listing 7: Various Integration Methods

```
MyInput = '0'
    while MyInput != "q":
        MyInput = input('Enter a choice, "1" for one
            dimensional square apeture (dblquad), "2" for two
             dimensional square apeture (dblquad), "3" for
            two dimensional circular apeture (dblquad), "4"
```

```
            for two dimensional circular apeture (Monte Carlo
            ), "b" to go back, or "q" to quit\n')
        if MyInput == '1':
            print('You have selected Part 1: One dimensional
                square apeture (dblquad)\n')
            Part = 1
            start_time = time.time()
                for xi in x:
                y = 0
                realpart, realerror = integrate.dblquad (
                    Fresnel2dreal, xmin ,xmax, ymin, ymax,
                    args=(y, xi, k, z), epsabs=1.49e-08,
                    epsrel=1.49e-08)
                imagpart, imagerror = integrate.dblquad (
                    Fresnel2dimag, xmin ,xmax, ymin, ymax,
                    args=(y, xi, k, z), epsabs=1.49e-08,
                    epsrel=1.49e-08)
                real_results.append(realpart)
                imag_results.append(imagpart)
                realerror_results.append(realerror)
                imagerror_results.append(imagerror)
            end_time = time.time()
            break
#============================================================
        elif MyInput == '2':
            print('You have selected Part 2: Two dimensional
                square apeture (dblquad)\n')
            Part = 2
            start_time = time.time()
            y, x = np.linspace(ymin_s, ymax_s, ylengthsteps)
                , np.linspace(xmin_s, xmax_s, xlengthsteps)
            for yi in y:
                for xi in x:
                    realpart, realerror = integrate.dblquad
                        (Fresnel2dreal, xmin ,xmax, ymin,
                        ymax, args=(yi, xi, k, z), epsabs
                        =1.49e-03, epsrel=1.49e-03)
                    imagpart, imagerror = integrate.dblquad
                        (Fresnel2dimag, xmin ,xmax, ymin,
                        ymax, args=(yi, xi, k, z), epsabs
                        =1.49e-03, epsrel=1.49e-03)
                    real_results.append(realpart)
                    imag_results.append(imagpart)
                    realerror_results.append(realerror)
                    imagerror_results.append(imagerror)
            end_time = time.time()
            break
#============================================================
        elif MyInput == '3':
            print('You have selected Part 3: Two dimensional
```

```
                circular apeture (dblquad)\n')
            Part = 3
            start_time = time.time()
            xmin, xmax, ymin, ymax = -R, R, -R, R
            y, x = np.linspace(ymin_s, ymax_s, ylengthsteps)
                , np.linspace(xmin_s, xmax_s, xlengthsteps)

            for yi in y:
                for xi in x:
                    realpart, realerror = integrate.dblquad
                        (Fresnel2dreal, xmin ,xmax, yminfunc,
                         ymaxfunc, args=(yi, xi, k, z),
                        epsabs=1.49e-03, epsrel=1.49e-03)
                    imagpart, imagerror = integrate.dblquad
                        (Fresnel2dimag, xmin ,xmax, yminfunc,
                         ymaxfunc, args=(yi, xi, k, z),
                        epsabs=1.49e-03, epsrel=1.49e-03)
                    real_results.append(realpart)
                    imag_results.append(imagpart)
                    realerror_results.append(realerror)
                    imagerror_results.append(imagerror)
            end_time = time.time()
            break
#============================================================
        elif MyInput == '4':
            print('You have selected Part 4: Two dimensional
                circular apeture (Monte Carlo)\n')
            Part = 4
            start_time = time.time()
            xmin, xmax, ymin, ymax = -R, R, -R, R
            y, x = np.linspace(ymin_s, ymax_s, ylengthsteps)
                , np.linspace(xmin_s, xmax_s, xlengthsteps)

            for y_ in y:
                for x_ in x:
                    results, results_error = MC(Fresnel2d_Mc
                        , xmin, xmax, ymin, ymax, x_, y_, k,
                        z, N)
                    E.append(results)
                    E_error.append(results_error)
            end_time = time.time()
            break
#============================================================
        elif MyInput == 'b':
            break
#============================================================
        elif MyInput != 'q':
            print('This is not a valid choice\n')
#============================================================
        elif MyInput == 'q':
```

```
            print('Program ending')
            sys.exit() #exit program
```

4.5 General Pre-plot calculations

A couple final things to do before the plots; first we need to convert our results lists into arrays for numpy vectorisation in our calculation using *np.asarray*, and then we need to calculate our intensities using our electric fields, and their errors using the partial differentials method, the Monte-Carlo method is calculated differently since it's field is in exponential form. We also calculate time taken for the method chosen by finding the difference in times. Lastly max error is found by calling the largest value from our error array and forcing it to be positive with *np.absolute*, and then using *np.round* to avoid the float from taking up most of the title. t is important to note that the error for Monte-Carlo is taken to less decimal places since it's error is much smaller than dblquad.

Listing 8: Intensity Calculations and Array creation

```
    real_results = np.asarray(real_results)
    imag_results = np.asarray(imag_results)
    realerror_results = np.asarray(realerror_results)
    imagerror_results = np.asarray(imagerror_results)
    E = np.asarray(E)
    E_error = np.asarray(E_error)
    if Part in (1, 2, 3):
        I = e0 * c * ( real_results**2 + imag_results**2 ) #
            W/m^2
        I_rel = I / I.max() # W/m^2
        I_error = np.sqrt (e0 * c * ( (2 * real_results *
            realerror_results )**2 + ( 2 *
        imag_results * imagerror_results )**2 ) ) # W/m^2
        I_errormax = np.round(np.absolute(I_error.max())),
            decimals = 11)
    else:
        I = e0 * c * (E * np.conjugate(E) ).real # W/m^2
        I_rel = I / I.max() #Relative value of Intensity # W
            /m^2
        I_error = E_error # W/m^2
        time_taken = end_time -start_time
        I_errormax = np.round(np.absolute(I_error.max()),
            decimals = 6)
```

4.6 Plotting our Results

Now we plot our graph depending on which part was chosen, we do this by creating an *if* statement, where separate parts of the code will run depending on what the *Part* value is, which was determined in the while loop above. For

part 1, we use a simple *plt.plot* with our Intensity against position, labelling our axes and creating an appropriate title listing the variables used, plotting our errors is done in the same way. For the 2D plots, we need to convert our result arrays into 2D so that they can be used in *plt.imshow*, so all we use is *np.reshape* and take care when considering the shape of our array (will depend on which variable is integrated first in the for loop). We define *extents* for *plt.imshow* which is just the smallest and largest values of each array in a specific order. Axes are labelled and the plot's title displays the variables used and time taken for the plot, as well as the maximum value of error in the Intensity during the integration. Part 3 is done the same way as part 2, and so is part 4 except that for the plot's title it includes N, the number of "guesses" made by the Monte-Carlo function.

Listing 9: Plotting our Results

```
 if Part == 1:
        plt.plot(x, I, label='Intensity')
        plt.xlabel("x (m)")
        plt.ylabel("Intensity (W/m^2)")
        plt.title('1D Dblquad Square-Apeture Diffraction;\n
            z = {} ,\n Time Elapsed = {:.2f} seconds, Apeture
             Width = {}' .format(z, time_taken, w))
        plt.show()

        plt.plot(x, I_error, color='red')
        plt.xlabel("x (m)")
        plt.ylabel("Error in the Intensity (W/m^2)")
        plt.title('Error in 1D Dblquad Square-Apeture
            Diffraction;\n z = {} ,\n Time Elapsed = {:.2f}
            seconds, Apeture Width = {}' .format(z,
            time_taken, w))
        plt.show()
#============================================================
    elif Part == 2:
        I_2D = np.reshape(I_rel, (ylengthsteps, xlengthsteps
            ))
        I_2derror = np.reshape(I_error, (ylengthsteps,
            xlengthsteps))
        extents = (x.min(), x.max(), y.min(), y.max())
        plt.imshow(I_2D,vmin=0.0,vmax=1.0*I_2D.max(),extent=
            extents,
        origin="lower",cmap="nipy_spectral_r")
        plt.xlabel("x (m)")
        plt.ylabel("y (m)")
        plt.title('2D Dblquad Square-Apeture Diffraction;\n
            Maximum Errror = {} W/m^2, Time Elapsed = {:.2f}
            seconds,\n z = {}, Apeture Width = {}' .format(
            I_errormax, time_taken, z, w))
        plt.colorbar()
        plt.show()
```

```
#============================================================
    elif Part == 3:
        I_2D = np.reshape(I_rel, (ylengthsteps, xlengthsteps
            ))
        I_2derror = np.reshape(I_error, (ylengthsteps,
            xlengthsteps))
        extents = (x.min(), x.max(), y.min(), y.max())
        plt.imshow(I_2D,vmin=0.0,vmax=1.0*I_2D.max(),extent=
            extents,\
        origin="lower",cmap="nipy_spectral_r")
        plt.xlabel("x (m)")
        plt.ylabel("y (m)")
        plt.title('2D Dblquad Circular-Apeture Diffraction;\
            n Maximum Errror = {} W/m^2, Time Elapsed = {:.2f
            } seconds,\n z = {}, Apeture Width = {}' .format(
            I_errormax, time_taken, z, w))
        plt.colorbar()
        plt.show()
#============================================================
    elif Part == 4:
        I_2D = np.reshape(I_rel, (ylengthsteps, xlengthsteps
            )).real
        I_2derror = np.reshape(I_error, (ylengthsteps,
            xlengthsteps))
        extents = (x.min(), x.max(), y.min(), y.max())
        plt.imshow(I_2D,vmin=0.0,vmax=1.0*I_2D.max(),extent=
            extents,\
        origin="lower",cmap="nipy_spectral_r")
        plt.xlabel("x (m)")
        plt.ylabel("y (m)")
        plt.title('2D Monte Carlo Circular-Apeture
            Diffraction;\n Maximum Errror = {} W/m^2, Time
            Elapsed = {:.2f} seconds,\n z = {}, Apeture Width
             = {}' .format(I_errormax, time_taken, z, w))
        plt.colorbar()
        plt.show()
```

4.7 Finishing Touches

We have a few final things to do before the project is complete; first we print the time taken for complete the chosen part to finish, as a reminder and also to confirm that the task has finished, then we create one final *for* loop to give the user an option to quit, or restart the entire program, similar to before.

Listing 10: Finishing Touches

```
print(f'Chosen Method took {time_taken:.2f} seconds./n')
    while True:
        MyInput = input('Type "r" to restart, "q" to quit/n'
            )
```

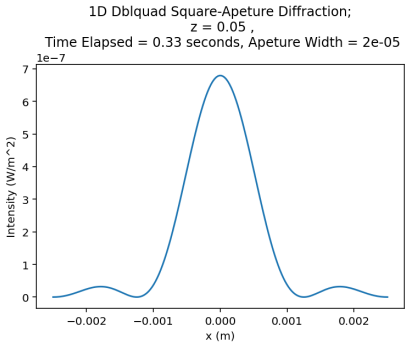
```
        if MyInput == 'r':
            break
        elif MyInput =='q':
            sys.exit()
        else: print('Invalid Input/n')
```

5 Results Analysis

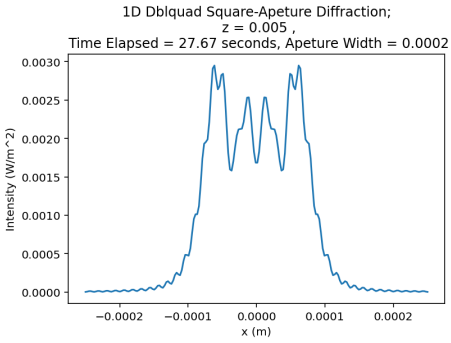
In this section I will review the plots generated from my code, and analyse the similarities and differences with using different aperture shapes, integration methods, and comparing different dimensions.

5.1 Part 1

Figure 1: Plots for part 1, 1D diffraction through a square aperture



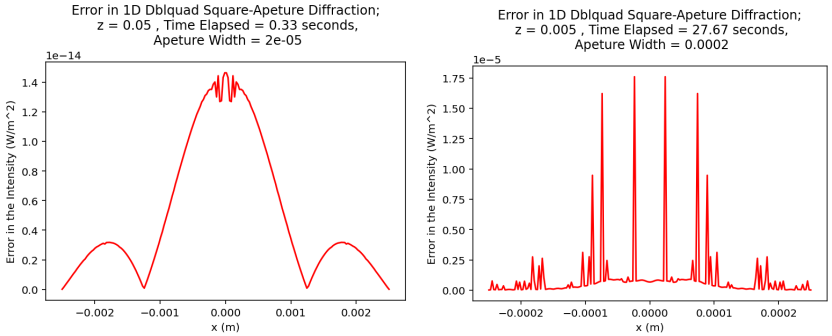
(a) Figure(1a) shows one dimensional Fraunhofer diffraction through a square aperture, where orders 1 and 2 peaks can be clearly seen at the $x = 0m$ and $x = \pm 0.0015m$ positions. The first order maxima is noticeably larger than the second orders, which aligns with expectations, as this is a standard Fraunhofer diffraction pattern, corroborating my simulation.



(b) Figure(1b) displays the error in dblquad for 1a, and largely agrees with physical expectations, as error is expected to rise with intensity. However the "dip" near $x = 0m$ is unexpected and explained below.

Figure 2: Errors in plots for part 1, 1D diffraction through a square aperture

Plots for part 1 : 1D diffraction through a square aperture



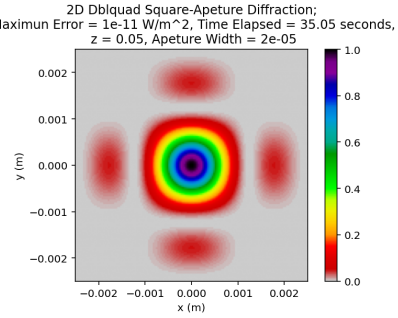
(a) Figure(2b) displays the error in dblquad for 1a, and largely agrees with physical expectations, as error is expected to rise with intensity. However the "dip" near $x = 0m$ is unexpected and explained below.

(b) Figure(2b) displays the error in dblquad for 1b, and agrees with physical expectations, as error spikes when intensity does, this looks different to 2a due to the nature of 1a since data spikes very sharply, so error is expected to too, and error for when intensity is decreasing is very small.

The plots for Fraunhofer and Fresnel diffraction can be seen in 1; the results coincide with the physical expectations; with Fresnel's large central pattern of varying intensity and Fraunhofer's spread out fringes of decreasing Intensity. The error in dblquad plots shown in 1 also agree with expectations, as the error increases sharply when there is a high intensity point, due to the increased amount of interference at that point. For the Fraunhofer plot in 2a, there is a small "dip" as the error approaches the central position symmetrically, this is likely due to how scipy's dblquad operates, as this would not be expected to appear for the error in the intensity, rather than the error in the integration function. The Fraunhofer error is significantly larger ($3 \times 10^{-3} W/m^2 >> 1.4 \times 10^{-14} W/m^2$) If our tolerances are varied. the time taken for the plot's change by an order of .01 seconds which is insignificant, and the shape of the curve does not change, however if we decrease $xlengthsteps$ and $ylengthsteps$ then the small "dip" towards $x = 0$ does reduce, as the precision in our plot becomes limited.

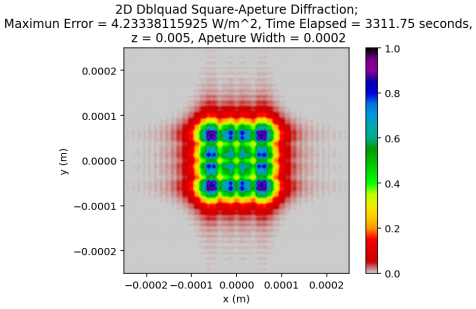
5.2 Part 2

Figure 3: Plots for part 2, 2D diffraction through a square aperture



(a) Figure(3a) shows my generated 2D Fraunhofer diffraction plot for a square aperture. Plot corresponds with physical expectation, with bright fringe in centre and dimmer second order fringes surrounding the central one. Max error in dblquad was found to be 1×10^{-11} which is very small, however still larger than in 2 1

as expected since Integration is in 2 dimensions now. Figure appears to be 2D version of 1 since parameters were the same, validating my simulation.

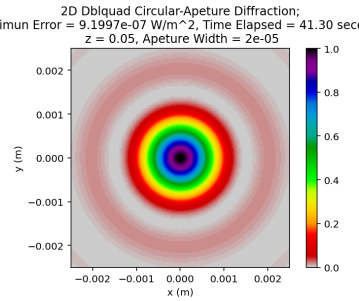


(b) Figure(3b) shows my generated 2D Fresnel diffraction plot for a square aperture, this plot agrees with the physical predictions of 4 brightest fringes in the corners, with dimmer maxima between them. Moreover, this graph appears to be literally a "2D" version of 1 which further validates this pattern. Time elapsed is 3312s, which is much larger than 3a since there is much more interference when the screen is close to the aperture, and so dblquad struggles to keep up.

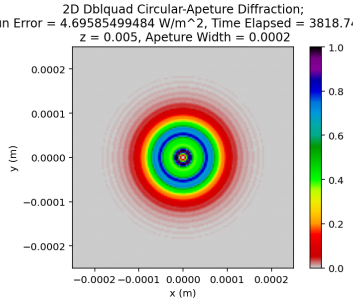
The plots for Fraunhofer and Fresnel diffraction can be seen in 3 , and hold parallels to the part one plots 1, which agrees with physical expectations since the parameters are kept the same. There are still subsequent orders of increasing intensity for Fraunhofer, and the maxima's further than $x = 0$ are larger than the ones closer. The time taken for the plot to generate is increased sharply when we introduce a second dimension, it goes up from less than a second for 1a and less than a minute for 1b, and increases to 62 seconds for Fraunhofer and 3312s for Fresnel! This time could be decreased by decreasing the resolution of the plot (*xlengthsteps* and *ylengthsteps*) but so that the figures in the report are nice and clear they were set to 150. However the relationship remains that calling dblquad for 2D takes much longer than 1D. The maximum error for Fraunhofer was $1 \times 10^{-11} W/m^2$, which is incredibly tiny, however 3 orders of magnitude larger than in 1a $1.4 \times 10^{-14} W/m^2$ due to the presence of a second dimension. Similarly for Fresnel, the error in 1b was $1.8 \times 10^{-5} W/m^2$, much smaller than the error in 3b which was 4 W/m^2, this is likely due to the large values of tolerances when the code was run, and so if I was prepared to wait longer, or reduce the resolution, then I could reduce this. The large error is due to the high interference when the screen is closer, similarly to why it takes longer to generate.

5.3 Part 3

Figure 4: Plots for part 3, 2D diffraction through a circular aperture



(a) Figure(4a) shows the plot for 2D Fraunhofer diffraction through a circular aperture. Maximum error was found to be $9 \times 10^{-7} W/m^2$, still very small yet significantly larger than in 3a, suggesting that it is harder for dblquad to integrate through a circular aperture. This is corroborated by the fact that the integration took 5 seconds longer than a square aperture also, suggesting that this pattern is harder for dblquad. The pattern looks similar to a square aperture, however appears completley circular which is to be expected.

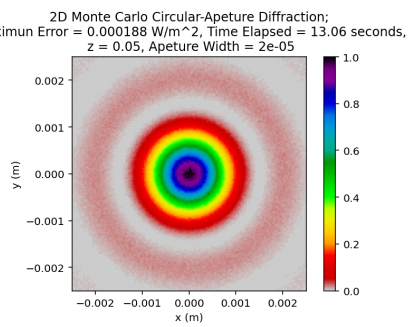


(b) Figure(4b) shows my 2D Fresnel Fresnel diffraction pattern, this looks completely circular alike 4a, showing that the pattern agrees with the expected physics. This pattern took much longer to complete, $3819s$ which is longer even than 3b implying that dblquad struggles more with a circular aperture than a square one, corroborating 4a. The max error here was $5W/m^2$, of the same magnitude as 3b which is to be expected since both are near-field. The pattern is very interesting, alike 3b, all the maximas are very close to the central fringe, With one faint one in the centre, a very intense one just outside it, and then a much larger lower intensity one around that, noting that all these appear around each other without intensity reading zero due to their close proximity to one another.

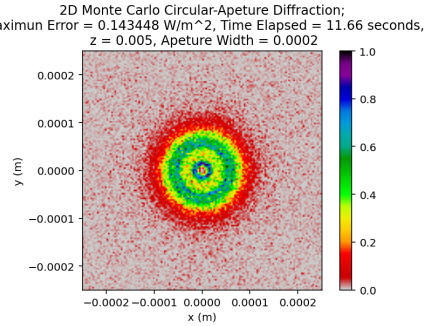
It is clear to see from 4 that diffracting through a circular aperture is much more difficult for dblquad since not only is the max error larger, but also is the time taken to generate. Also both patterns appear more circular, which makes sense due to them being diffracted through a square aperture rather than the square-like properties seen in 3 but more obiously 3b.

5.4 Part 4

Figure 5: Plots for part 4, 2D diffraction through a circular aperture using Monte-Carlo, N = 10000



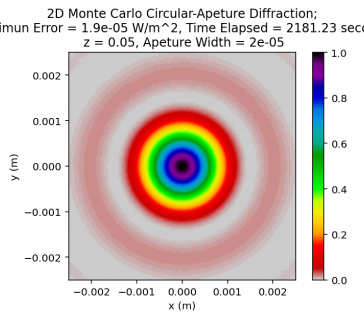
(a) Figure(5a) displays my pattern for 2D Fraunhofer circular diffraction using the Monte-Carlo method, specifically using $N = 10,000$. This method produced a pattern in significantly less time than 4a, only 13s, due to the nature of the Monte-Carlo method this is to be expected, as it essentially "guesses" what the function will look like with an accuracy proportional to the size of N. However because of the "guess" work, the error increased sharply to $2\times10^{-4}W/m^2$ essentially making this method great for a quick idea of the pattern but poor for high accuracy. For far-field, the plot is easy to guess even at low N and so this pattern looks indistinguishable to 4a.



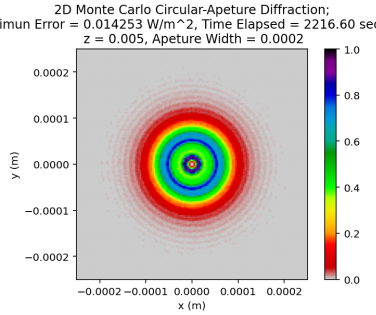
(b) Figure(5b) displays my Fresnel diffraction pattern through a circular aperture using the Monte-Carlo method, using $N = 10,000$. This method produced a pattern far faster than when using dblquad (only 12s!), however due to the complicated nature of near-field diffraction, the current N value is far too low to get a plot that looks like 4b; indeed increasing our N value should produce a graph that more accurately depicts what is seen using dblquad. Interestingly enough, the max error is an order of magnitude lower than in 4b at $0.1W/m^2$, likely because the Monte-Carlo error is not the error in the function, but the actual error in Intensity and so should not be compared with the dblquad errors. This number is still higher than 5a which is to be expected for a Fresnel pattern. Despite the high error though, the high intensity central maxima and the slightly lower intensity ring of diameter $\pm5 \times 10^{-5}$ can still be seen here, alike 4b suggesting that this pattern still agrees with what is predicted to happen, we just need to increase N tp get a clearer plot.

It is clear to see in 5 that the MC method is significantly faster, however struggles to produce more accurate plots for Fresnel diffraction unless N is very large. Despite this though, the method agrees with what is predicted to happen and so can be considered a success for using Monte-Carlo. However to validate my expectation for 5b becoming more clear as N increases, I will test this in 6 (and by waiting a very long time!).

Figure 6: Plots for part 4, 2D diffraction through a circular aperture using Monte-Carlo, N = 1 million



(a) Figure(6a) shows my Fraunhofer diffraction patter through a circular aperture using the Monte-Carlo method, with $N = 1million$. As expected the pattern looks more similar to 4 than 5 does despite the difference being small but noticeable at high intensities, due to the N amount corresponding to a higher number of "guesses" made. This plot took $2181s$ to generate, so although we get a better plot, the time taken makes it not worth the time, especially due to the small differences in this plot. Also the error here, $1.9 \times 10^{-5} W/m^2$, is a lot smaller than in 5 due to the plot being more "certain" it is a good approximation, as we have a high N value.



(b) Figure(6b) shows my Fresnel diffraction pattern through a circular aperture using the Monte Carlo method, at $N = 1million$. As expected the plot is far clearer and a far better approximation of 4 than 5 is. It is more noticeable here than 6a since the higher interference in Fresnel diffraction makes it harder for the Monte-Carlo method to "guess" what the pattern will look like. Similarly to 6a the time taken has risen, here to $2217s$ and the error has decreased, to $0.01 W/m^2$ since the plot has taken more time to be "more sure" of the pattern. Despite this however, due to the efficiency of the Monte-Carlo method compared to dblquad, this still took less time, and has produced a near identical pattern.

As expected for increasing the N count, 6 takes far more time than 5, and far less than 4 since although more "guesses" has been taken, its still an approximation. Also, the errors in the intensity are much smaller in 6 than for 5 due to the increased number of N reducing the error. 6 shares striking similarities to 4 which is to be expected, as 6's Monte-Carlo method basically approximated 4, and so we can expect with higher number of guesses N, the closer you get to the true physical pattern.

6 Conclusion

Overall, I successfully created a program which simulates diffraction through an aperture through both a circular and square slit using equations 1, 2, 3, and 4. I found that simulation a diffraction pattern in 1D too far less time and produced a far smaller error than in 2D, since it meant less strain on dblquad due to the system being more simple. Moreover Fresnel diffraction consistently took more time than Fraunhofer which aligns with physical expectations as there is more interference when the screen is closer to the aperture. Additionally, making the aperture circular took more time and produced a higher error than

for when the aperture was square, this was likely due the fact that for a circular aperture, each y coordinate is a function of x, forcing more calculations in each iteration of my for loop, so it makes sense that the time taken for the integration would increase, as well as the error also increasing. I also varied the method used to simulate the diffraction patterns from "dblquad" to "Monte-Carlo" and found that using the dblquad works by showing exactly what would happen by integrating the Fresnel equation, whereas the Monte-Carlo method essentially "approximates" the predicted diffraction pattern using the area over the integral and the average value of the function. I found that, since the Monte-Carlo is essentially an approximation, it takes less time to compute, however this time is increased with increasing N, with the error decreasing as well, noticing how N is the denominator in the fraction describing the error in 4. I increased N from $10k$ to $10million$ and found that whilst there was little change in the Fraunhofer diffraction pattern, due to the limited interference going on when the screen is relatively far away from the aperture, but for Fresnel diffraction, a large value of N was needed for the Monte-Carlo method to meaningfully replicate 4. All of this analysis for my simulation shows that I successfully recreated expected diffraction patterns on a computer, and was able to use this to study the physics behind it by easily tweaking the parameters and figuring out which method was the most optimal, which I found was the Monte-Carlo method, providing that the N value was not too large, around $100k$ should do, as the plot doesn't have to be perfect but it has to be good enough to accurately see what's going on.